UANL
UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN

NOMBRES Y MATRICULAS

2251959	CASTRO ALVAREZ JACK LEVI
2250780	GOMEZ VELAZQUEZ WILLIAM
2250824	GONZALEZ MEJIA CARLOS ADRIAN
2251387	HERNÁNDEZ VALERO JESÚS DANIEL

AULA	GRUPO	TURNO
144	5V1	MATUTINO

ESPECIALIDAD

Técnicos en Inteligencia Artificial BPI

DOCENTE

Jesus Alejandro Juarez Gonzales

FECHA

MONTERREY, NUEVO LEÓN A DOMINGO 8 DE NOVIEMBRE DEL 2026

1. El problema

Ferretería Los Nogales (empresa ficticia) entrega material en obras del área de Monterrey. Sus clientes piden por notas de voz de WhatsApp y el personal las captura a mano, con errores en cantidades y medidas. **Receipt** convierte una nota de voz en español en un ticket con cinco campos: **cliente, producto, cantidad, medida y hora de entrega**.

El sistema tiene dos pasos. Primero, **Whisper** transcribe el audio a texto. Después, un **extractor** lee la transcripción y llena el ticket. La medida no se adivina: sale del producto del catálogo de 42 productos (`data/catalog.csv`).

Datos y división

El equipo grabó 30 notas de voz con pedidos actuados, con distintas voces, velocidades y niveles de ruido. Tres eran copias idénticas de otra nota y se quitaron para que una misma nota no quedara en entrenamiento y en prueba a la vez. Quedaron **27 notas**, cada una con su transcripción y su ticket correcto en `data/notes.csv`.

Antes de evaluar, las notas se dividieron en **16 de entrenamiento, 6 de validación y 5 de prueba** (60/20/20, semilla 42), con las cuatro personas y los tres niveles de ruido en la prueba. Los dos modelos se evalúan sobre las **mismas 5 notas de prueba**.

2. Modelo base: Whisper small + reglas

La línea base usa herramientas de las primeras etapas del curso. `openai/whisper-small` transcribe el audio, forzando el idioma español. Después, unas **reglas con expresiones regulares** buscan en el texto:

- el **producto**, comparando el texto con los sinónimos del catálogo ("cable del doce" corresponde a `CAB-003`);
- la **hora**, con patrones como "para las tres de la tarde" o "3:30 pm";
- la **cantidad**, que es el número que queda cuando se quitan la hora y el producto;
- el **cliente**, que es lo que sigue a "soy", "habla" o "de parte de".

Antes de aplicar las reglas, el texto se normaliza: se pasa a minúsculas, se quitan los acentos y los números se escriben con dígitos ("cincuenta" pasa a 50). Así, "cable del doce" y "cable del 12" se leen igual. Elegimos reglas porque son rápidas, no necesitan red ni GPU, y se puede explicar exactamente por qué dieron cada resultado.

```
def extract(text, catalog):
    normalized = normalize_text(text)        # minúsculas, sin acentos, dígitos
    matches = catalog.find_all(normalized)     # sinónimos del catálogo
    hora, normalized = extract_time(normalized)
    ...
    ticket = Ticket(cliente=extract_client(text),
                    cantidad=extract_quantity(remaining), hora_entrega=hora)
    return ticket.with_product(matches[0].product) if matches else ticket
```

receipt/extraction/rules.py — extractor de la línea base (resumido)

3. Modelo preentrenado sin ajuste

El segundo modelo usa Transformers preentrenados **tal como vienen, sin fine-tuning**:

- **Transcripción:** `openai/whisper-medium`, un Whisper más grande que el de la línea base. Whisper es un Transformer encoder-decoder con atención: el encoder procesa el audio y el decoder escribe el texto palabra por palabra, atendiendo a las partes del audio que le sirven.
- **Extracción:** el modelo de lenguaje `qwen3:4b-instruct` corre localmente con Ollama. Recibe la transcripción y el catálogo, y devuelve el ticket en JSON. Usa `temperature` 0 para que siempre responda igual, y un esquema JSON que obliga a devolver los cuatro campos.

Si Ollama no responde, el sistema usa las reglas como respaldo y lo marca en el resultado, para que la demo nunca se caiga. En la evaluación no fue necesario usar el respaldo en ninguna nota.

```
payload = {"model": OLLAMA_MODEL,
           "messages": build_messages(text, catalog),   # prompt + catálogo
           "format": SCHEMA,                            # obliga a responder en JSON
           "options": {"temperature": 0}}
reply = call_ollama(payload)
data = json.loads(reply["message"]["content"])
ticket = ticket_from_data(data, catalog, text)
```

receipt/extraction/llm.py — extracción con el modelo de lenguaje

Los dos modelos se configuran en un solo lugar y se eligen con `--mode`:

```
MODES = {
    "baseline":   {"whisper": "openai/whisper-small",  "extractor": "rules"},
    "pretrained": {"whisper": "openai/whisper-medium", "extractor": "llm"},
}
```

receipt/config.py (resumido)

4. Métrica principal

El reto define dos métricas principales, una para cada paso del sistema. Además medimos el tiempo de cada modelo.

WER (word error rate)

Mide qué tan bien transcribe Whisper. Cuenta cuántas palabras hay que sustituir (S), borrar (D) o insertar (I) para convertir la transcripción del modelo en la correcta, y lo divide entre el número de palabras de la correcta (N): **WER = (S + D + I) / N**. Un WER **más bajo es mejor**. Antes de calcularlo, los dos textos se normalizan igual, para que la puntuación o escribir "50" en vez de "cincuenta" no cuenten como error.

Porcentaje de campos correctos

Mide qué tan bien queda el ticket. Para cada nota se comparan los cinco campos con el ticket correcto, y se reporta el porcentaje total y el de cada campo. **Más alto es mejor.** El cliente se compara sin distinguir mayúsculas ni acentos, la cantidad como número y la hora en formato HH:MM.

```
def word_error_rate(references, hypotheses):
    refs = [normalize_text(ref) for ref in references]
    hyps = [normalize_text(hyp) for hyp in hypotheses]
    return jiwer.wer(refs, hyps)

def score_fields(reference, ticket):
    return {
        "cliente": _fold(reference["cliente"]) == _fold(ticket.cliente),
        "cantidad": _as_number(reference["cantidad"]) == _as_number(ticket.cantidad),
        ...
    }
```

receipt/evaluation.py — cálculo de las métricas (resumido)

Tiempo de procesamiento

Se mide, para cada nota, cuántos segundos tarda la transcripción y cuántos la extracción. Es lo que importa en la prueba del juez: los segundos desde que termina el audio hasta que aparece el ticket. La carga de los modelos se reporta aparte, porque ocurre una sola vez al iniciar.

5. Resultados en el conjunto de prueba

Los dos modelos corrieron sobre las mismas 5 notas de prueba, en una GPU NVIDIA RTX 5050. Las cifras salen de `results/` en el repositorio.

Modelo	WER	Campos correctos	Tiempo por nota (media)	Tiempo p95
Base: Whisper small + reglas	18.3 %	60.0 %	4.69 s	7.55 s
Preentrenado: Whisper medium + qwen3:4b	12.7 %	76.0 %	11.66 s	18.11 s

Campo	Base	Preentrenado
Cliente	80 %	60 %
Producto	20 %	100 %
Cantidad	80 %	40 %
Medida	20 %	100 %
Hora de entrega	100 %	80 %

6. Conclusión

El modelo preentrenado es más preciso: transcribe mejor (WER de 12.7 % contra 18.3 %) y llena bien más campos (76 % contra 60 %). La gran diferencia está en **producto y medida**. Las reglas solo encuentran el producto si el texto trae un sinónimo exacto del catálogo, y Whisper small escribió "blog

de concreto" en vez de "block" e "impermeabilidad santa" en vez de "impermeabilizante". El modelo de lenguaje sí entendió esos productos.

Las reglas ganan en cliente, cantidad y hora. El modelo de lenguaje dejó títulos en el nombre ("don José"), tomó la cantidad de otro producto de la misma nota y convirtió "10 p.m.", que Whisper medium escuchó así, en 22:00. Además, la línea base es más de dos veces más rápida (4.69 s contra 11.66 s por nota).

Límites: la prueba tiene solo 5 notas, así que cada nota vale 20 puntos en cada campo, y la transcripción de referencia todavía está en revisión. Para la Fase 2 la idea es combinar lo mejor de cada uno: el modelo de lenguaje para producto y medida, y las reglas para hora y cantidad.